

Usage-Centered Software Engineering: An Agile Approach to Integrating Users, User Interfaces, and Usability into Software Engineering Practice

Larry L. Constantine
University of technology, Sydney (Australia)
Constantine & Lockwood, Ltd.
lconstantine@foruse.com

Lucy A. D. Lockwood
Constantine & Lockwood, Ltd.
llockwood@foruse.com

Abstract

Usage-centered design is a systematic, model-driven approach to visual and interaction design with an established record of effectiveness in a wide variety of settings and areas of application. The tutorial introduces the models and methods of usage-centered design and explores the integration of usage-centered approaches into software engineering practice. Agile approaches to modeling will be emphasized, with the focus on use cases, which are central to usage-centered design and serve as a common thread throughout an integrated usage-centered software engineering process.

1. Introduction

In the final analysis, nearly all software is intended to serve the needs and support the interests of its human users, yet software engineering is often weakest when it comes to addressing the critical areas of user requirements, usability, user interfaces, and interaction design. Often these concerns are either minimized, ignored, or relegated to "other" disciplines as a responsibility outside the scope of software engineering. This tutorial shows how user requirements, usability, and user interface design can be made the very core of a successful software engineering process.

Usage-centered design [1, 2] is a systematic, model-driven approach to visual and interaction design for user interfaces in software and Web-based applications. On projects of widely varying scope and scale in a variety of application areas [2, 3, 4, 5, 6], usage-centered design has proved capable of delivering superior designs [7]. As the name suggests, usage-centered design differs from conventional user-centered approaches primarily in a shift of focus from users per se to usage, that is, to the tasks to be accomplished by users. This difference in emphasis is reflected in differing practices. Instead of trial-and-error

design through repeated cycles of prototyping, user feedback, and usability testing, usage-centered design constructs robust and highly refined abstract models and more or less directly derives from these an initial design requiring only limited usability testing and minimal refinement.

Usage-centered design integrates readily with software engineering precisely because it is grounded in a strong engineering orientation and was developed from the outset to be compatible with object-oriented software engineering [9]. The very same extensions and refinements to well established models and techniques, such as actors and use cases [8, 9], that drive the user interface design can also be employed directly for software engineering [10].

2. Agile Methods and Usability

Extreme programming [11] and other of the emerging agile methods [12] represent an important new direction in software engineering and development. The emphasis in all the agile methods is on accelerated but highly disciplined development of high-quality software based on close coordination with customers or customer representatives and minimal analysis and design artifacts. As a body of practice, agile methods have a track record on usability and user interface design that is no better than traditional large-scale software engineering methods. However, the adaptability of both usage-centered design and the agile methods has made possible their successful integration [4].

The tutorial will introduce the models and methods of usage-centered design with an emphasis on agile modeling and design techniques [2, 13, 14, 15], both for their inherent simplicity and for their advantages as a framework for hands-on learning. The integration of usage-centered approaches into software engineering practice will be considered, with special emphasis on task cases (or essential use cases) [8, 16], which are central to

usage-centered design and serve as a common thread tying together the entire usage-centered software engineering process [8, 10].

A usage-centered task model differs from both use cases and ubiquitous scenarios in providing a much simpler and finer-grained representation of the elemental structure of work being accomplished by users. This simplicity and granularity enables more comprehensive modelling, facilitates validation of models, and promotes reuse of design elements and components in the software engineering process.

3. References

- [1] Constantine, L. L., and Lockwood, L. A. D. *Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design*. Addison-Wesley, Reading, MA, 1999.
- [2] Constantine, L. L., and Lockwood, L. A. D. "Usage-Centered Engineering for Web Applications. *IEEE Software*, 19 (2), March/April. 2002.
- [3] Anderson, J., Fleek, F., Garrity, K., and Drake, F. "Integrating Usability Techniques into Software Development. *IEEE Software*, 18 (1), January/February, 2001.
- [4] Patton, J. "Extreme design: Usage-Centered Design in XP and Agile Development." In L. Constantine (Ed.), *forUSE 2002: Proceedings of the First International Conference on Usage-Centered, Task-Centered, and Performance-Centered Design*. Ampersand Press, Rowley, MA, 2002.
- [5] Strobe, J. "Putting Usage-Centered Design to Work: Clinical Applications." In L. Constantine (Ed.), *forUSE 2002: Proceedings of the First International Conference on Usage-Centered, Task-Centered, and Performance-Centered Design*. Ampersand Press, Rowley, MA, 2002.
- [6] Windl, H. "Designing a Winner: Creating STEP 7 Lite with Usage-Centered Design." In L. Constantine (Ed.), *forUSE 2002: Proceedings of the First International Conference on Usage-Centered, Task-Centered, and Performance-Centered Design*. Ampersand Press, Rowley, MA, 2002.
- [7] Windl, H., and Constantine, L. "Performance-Centered Design: STEP 7 Lite." Winning submission, Performance-Centered Design 2001, <http://foruse.com/pcd/>
- [8] Constantine, L. L., and Lockwood, L. A. D. "Structure and Style in Use Cases for User Interface Design." In M. van Harmelan (Ed.), *Object Modeling and User Interface Design*. Addison-Wesley, Boston, 2001.
- [9] Jacobson, I., Christerson, M., Jonsson, P., and Övergaard, G. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Reading, MA, 1992.
- [10] Biddle, R., Noble, J., and Tempero, E. "From Essential Use Cases TO Objects." In L. Constantine (Ed.), *forUSE 2002: Proceedings of the First International Conference on Usage-Centered, Task-Centered, and Performance-Centered Design*. Ampersand Press, Rowley, MA, 2002.
- [11] Beck, K. *Extreme Programming Explained*. Addison-Wesley, Reading, MA, 2000.
- [12] Cockburn, A. *Agile Software Development*. Addison-Wesley, Boston, 2002.
- [13] Ambler, S. *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*. Wiley, New York, 2002.
- [14] [Constantine, L. L. "Cutting Corners: Shortcuts in Model-Driven Web Design." *Software Development*, 8 (2). Reprinted in L. Constantine (Ed.), *Beyond Chaos: The Expert Edge in Managing Software Development*. Addison-Wesley, Boston, 2001.
- [15] Constantine, L. L. "Process Agility and Software Usability: Toward Lightweight Usage-Centered Design." *Information Age*, 8 (2). Reprinted in L. Constantine (Ed.), *Beyond Chaos: The Expert Edge in Managing Software Development*. Addison-Wesley, Boston, 2001.
- [16] Constantine, L. L. (1995) "Essential Modeling: Use Cases for User Interfaces," *interactions* 2 (2).